SEGFAULT

Line-Rate DNS Flood Defense with eBPF / XDP

Trust-tiered, cost-weighted admission control in the kernel datapath

"Rival designs ask how fast. Segfault asks who, and what it costs."

Problem Statement: PS-022 — eBPF/XDP Rate Limiting for DNS Query Floods in a Lab
Category: Cybersecurity

Team: Segfault Team code: A3-PS022-TC005
Institution: Institute of Engineering & Management (IEM), Kolkata

Debangan Mali · **Saraswata Chatterjee** · **Partha Chakraborty** (Faculty)
debanganmali99@gmail.com

28 October 2026

Controlled-laboratory system. All attack traffic was generated against isolated virtual machines owned by the authors.

Table of Contents

Table of Contents 2
1. Problem Description 3
2. Solution Proposed 3
  2.1 Concept — what Segfault does differently 3
  2.2 How it is executed 3
3. Optimization Proposed by the Team 5
  3.1 Before / after comparison 5
  3.2 The optimization process 5
  3.3 List of optimizations 6
4. Solution Architecture and Design 7
  4.1 Admission control — the mathematics 7
  4.2 Implementation and kernel constraints 9
  4.3 Admission pseudocode 9
5. Performance Evaluation and Comparison 10
  5.1 Methodology 10
  5.2 IPv4 results — all six scenarios (median of 3) 10
  5.3 IPv6 results (median of 3) 10
  5.4 Water-torture at three realism levels (segfault vs best baseline) 10
  5.5 Result integrity 11
  5.6 Throughput ceiling 11
  5.7 Comparison with alternative approaches 12
  5.8 Honest scope and limitations 12
6. References 13
7. Conclusion 13

1. Problem Description

High-volume UDP DNS query floods disable a resolver not by brute bandwidth alone, but by forcing the operating system to spend CPU before a packet is ever useful. For every incoming query the kernel allocates a socket buffer, traverses the network stack, and wakes the resolver — work that is spent even on packets that are pure attack. The eXpress Data Path (XDP) runs an eBPF program inside the NIC driver, before that allocation, so an unwanted packet can be dropped at the earliest possible point. PS-022 asks for an XDP filter that applies bounded DNS rate limits and measures the effect on legitimate service.

The problem is harder than "drop more packets," because at the wire the attacker's packets and the victim's packets are nearly identical. The key issues:

- **Spoofable source addresses.** UDP has no handshake, so an attacker forges arbitrary source IPs — any defence that trusts the source address is defeated.
- **The adaptive attacker.** A fixed per-source threshold is defeated by spreading the flood over many sources, each kept just under the limit, while the aggregate still saturates the server.
- **Unequal query cost.** An ANY query, a DNSSEC-signed answer, or a random-subdomain (water-torture) flood costs far more server work than a single A lookup.
- **Collateral damage.** Naive per-prefix limiting punishes a legitimate client that happens to share a /24 with spoofed sources.
- **Reflection risk.** Any reply the defence emits toward a spoofed source is a potential reflection toward a victim.

Scope (as mandated by PS-022): a single controlled host-pair lab; bounded (not full-semantic) DNS parsing; no public flood testing; no claim of universal random-subdomain detection; and no claim of eliminating reflection — Segfault *bounds* reflection, it does not eliminate it.

2. Solution Proposed

2.1 Concept — what Segfault does differently

A fixed-threshold limiter reduces every client to a single number: its packet rate — which is exactly the quantity the attacker controls. Segfault replaces the single question "how fast?" with two:

- **Who is this?** Each source is placed in a trust tier assigned by proof of address, not by a forgeable source IP.
- **What does it cost?** Each query is charged a weight proportional to the server work it induces.

Capacity is then allocated as a reservation: clients that have cryptographically proven they are not spoofed draw from a pool that anonymous traffic can never touch. An attacker who cannot complete a round-trip can never enter that pool, no matter how many sources it forges or how it paces them.

2.2 How it is executed

Three cooperating components implement the concept:

- **XDP ingress program** (~3,010 verified instructions) — parses each DNS packet on a per-CPU scratch buffer, classifies its trust tier, charges its cost against token buckets, and returns XDP_PASS, XDP_DROP, or XDP_TX (a truncated reply).
- **tc-egress observer** (~644 instructions) — watches outgoing responses to learn which clients the server actually answered (a proof signal) and to count NXDOMAIN/SERVFAIL per zone.

- **Userspace controller** — a 500 ms control loop that moves the system between CALM / GUARD / SIEGE modes, rotates the sketch epoch, and reprices zones with heavy NXDOMAIN. Persistent state lives in pinned BPF maps, so the datapath survives a controller restart.

Trust tiers

Tier	Name	How it is entered
T0	Unknown	Default for any unseen source. No per-source state; volume tracked statelessly with a Count-Min sketch. Draws from a small, tightly capped pool.
T1	Returning	A source seen behaving politely, promoted under a token-bucketed promotion rate. Gets a per-source allowance.
T2	Proven	A source that proved its address — by a TCP query or a valid RFC 9018 DNS cookie. Draws from a reserved pool floods cannot enter.

The asymmetry is deliberate: entering T2 needs a round-trip the attacker cannot fake with a spoofed source, so the protected pool is structurally unreachable by spoofed traffic.

3. Optimization Proposed by the Team

Starting from the baseline required by PS-022 (a fixed-threshold XDP limiter, representative of rival designs), the team introduced five optimizations. Each targets a specific way the fixed-threshold design fails.

3.1 Before / after comparison

Dimension	Before — fixed threshold (rivals)	After — Segfault
Decision input	Source packet rate only	Trust tier (by proof) + query cost
Spoofed sources	Trusted as distinct clients	Confined to T0; never reach the proven pool
Adaptive flood	Evades the per-source limit	Caught by drop-fraction escalation + sketch
Query cost	All queries equal	Cost-weighted: ANY/DO/large/NXDOMAIN zones cost more
Prefix-spoof	Punishes the real client	Admits on proof, not prefix — client survives
Dropped excess	Silent drop (client times out)	1-in-k truncated TC reply → legit retries over TCP
Aggression	Static	Adaptive CALM/GUARD/SIEGE with hysteresis

3.2 The optimization process

The admission decision below shows how the optimizations compose into a single bounded pipeline evaluated per packet:

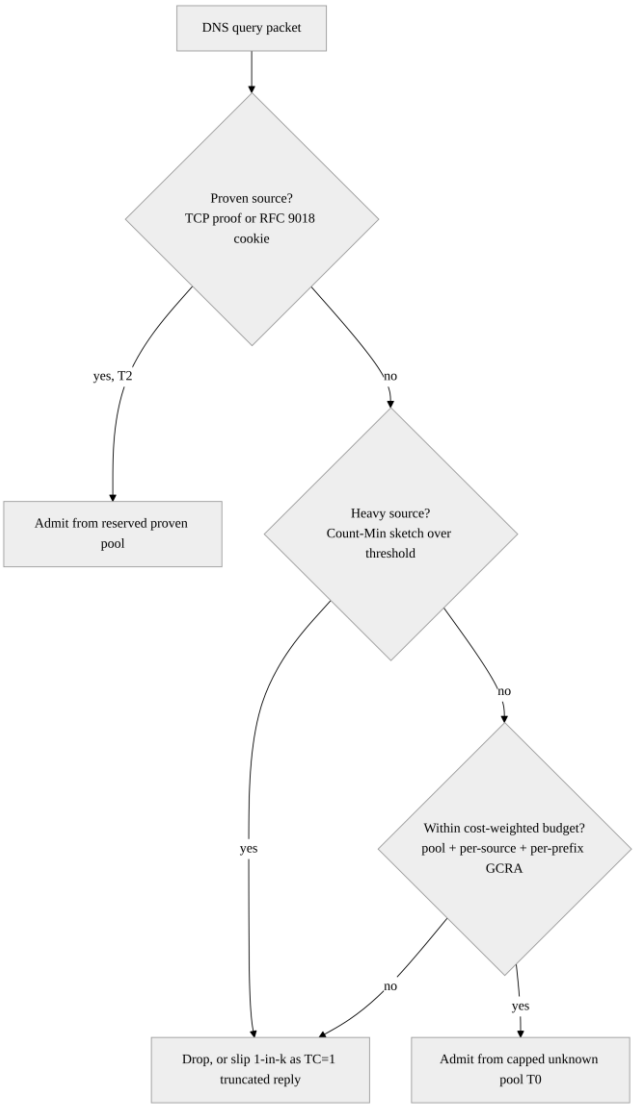


Figure 1. The cost-weighted, trust-tiered admission decision evaluated for each DNS query.

3.3 List of optimizations

- **Trust tiers by proof** — TCP/cookie proof promotes a client to a reserved pool spoofed floods cannot enter.
- **Cost-weighted admission** — GCRA token buckets drain faster for expensive queries; the controller reprices high-NXDOMAIN zones.
- **Stateless T0 tracking** — a Count-Min sketch bounds memory under a flood of millions of spoofed sources.
- **Bounded reflection (TC-slip)** — a 1-in-k truncated reply caps reflected volume at 1/k and removes amplification.
- **Adaptive controller** — drop-fraction + response-health drive CALM/GUARD/SIEGE with escalate-fast / de-escalate-slow hysteresis.

4. Solution Architecture and Design

Figure 2 shows the full datapath. A packet enters the XDP hook, is parsed on a per-CPU scratch buffer, classified into a tier, charged its cost against three token buckets in series, and then passed, slipped, or dropped. Responses leaving the box are observed on the tc-egress hook, which feeds the userspace controller; the controller writes back the mode, per-zone costs, and sketch epoch.

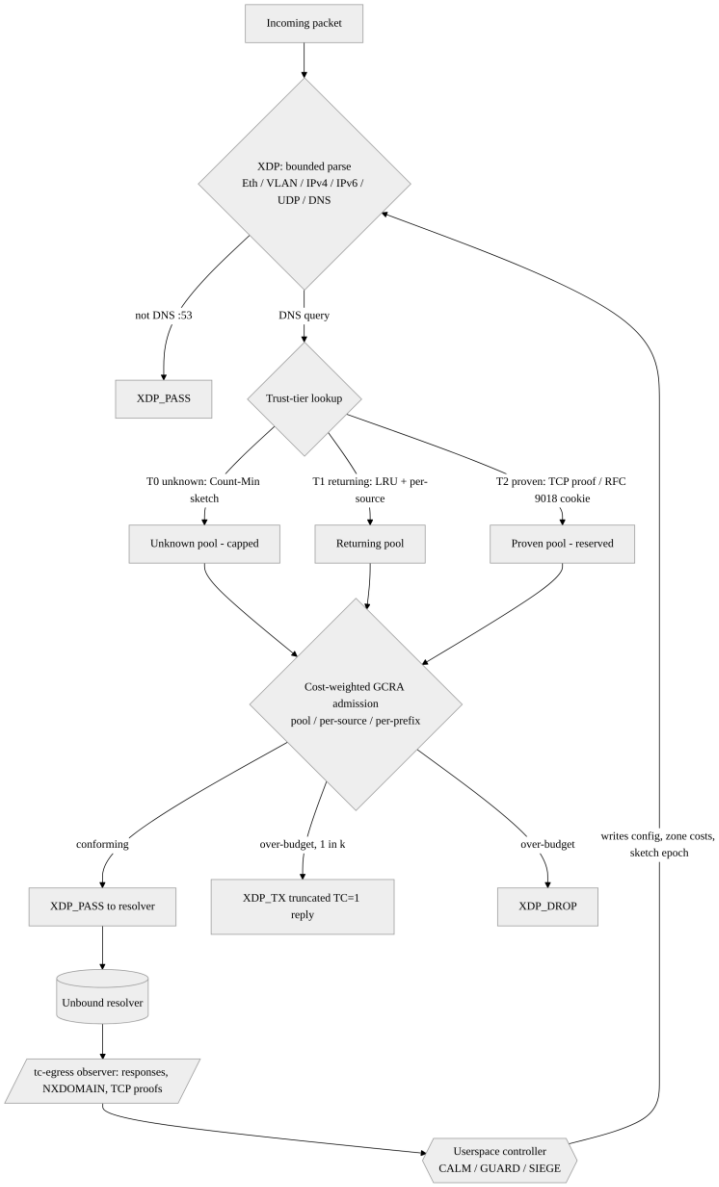


Figure 2. Segfault architecture: XDP ingress datapath, tc-egress observer, and the userspace controller feedback loop.

4.1 Admission control — the mathematics

GCRA token buckets

Admission uses the Generic Cell Rate Algorithm (GCRA), implemented lock-free with a compare-and-swap on a 64-bit nanosecond budget. A bucket has increment T (emission interval) and tolerance τ (burst):

$$T = 10^9 / \lambda \quad \text{ns per unit}, \qquad \tau = T \cdot B$$

where λ is the permitted rate (queries/second) and B the burst (units). The bucket stores a theoretical arrival time (TAT). A query of cost w arriving at time t_a is admitted iff

$$t_a \geq TAT - \tau,$$

and on admission the TAT advances by the cost-scaled increment:

$$TAT \leftarrow max(t_a, TAT) + w \cdot T.$$

A rejected query does not advance the bucket, so a rejected flood cannot push the bucket into the future and starve later legitimate queries. Each query is checked against the tier pool, the per-source bucket, and the per-prefix (/24 or /48) bucket, and is admitted only if all three conform.

Capacity allocation

With measured capacity C and proven/unknown shares σ_p, σ_u:

$$\lambda(T2) = C{\cdot}\sigma_p \quad ; \quad \lambda(T0) = (C - \lambda(T2)){\cdot}\sigma_u \quad ; \quad \lambda(T1) = max((C - \lambda(T2))(1-\sigma_u), 0.05{\cdot}C)$$

With lab defaults C = 20,000, $\sigma_p = 0.5$, $\sigma_u = 0.3$ this reserves 10,000 q/s for proven clients and caps unknown traffic at 3,000 q/s — 15% of capacity — before any attack begins. In SIEGE the unknown pool and per-source/prefix rates are scaled to 0.25× / 0.5× / 0.5× while the proven pool is left untouched.

Stateless T0 estimation (Count-Min sketch)

Per-source state for millions of spoofed addresses would itself be a memory-exhaustion vector. T0 volume is tracked in a Count-Min sketch of d hash rows × w counters; the estimate for source x is

$$\hat{c}(x) = min \text{ over } j \text{ of } C[j][\, h_j(x) \bmod w \,]$$

The sketch never under-counts, and with $w = \lceil e/\varepsilon \rceil$, $d = \lceil \ln(1/\delta) \rceil$ the over-count is bounded by $\hat{c}(x) \leq c(x) + \varepsilon N$ with probability $\geq 1-\delta$, where N is the window total. Cost is O(d) memory touches per packet, independent of the number of distinct spoofed sources.

Cost weighting

Each query is charged a multiplicative cost

$$w = w_0 \cdot m_ANY^{[qtype=ANY]} \cdot m_DO^{[DO\ bit]} \cdot m_big^{[large\ qtype]} \cdot z(zone)$$

where each exponent is a 0/1 indicator and z(zone) ≥ 1 is raised by the controller for zones whose NXDOMAIN ratio exceeds 0.2 (×2) or 0.5 (×max(m_big,4)) — the signature of a random-subdomain flood. Defaults: m_ANY = 8, m_DO = 2, m_big = 3.

Cryptographic proof of address (RFC 9018 cookies)

The only non-TCP path into the proven tier is a valid RFC 9018 server cookie, a keyed PRF (SipHash-2-4) over the client cookie, version, reserved bytes, timestamp, and client IP, under a server secret k:

$$SC = SipHash\text{-}2\text{-}4_k(\, client\text{-}cookie \,\|\, version \,\|\, reserved \,\|\, timestamp \,\|\, client\text{-}IP \,)$$

Because the client IP is an authenticated input and the secret never leaves the server, a cookie minted for one address is invalid from any other, and cannot be forged for a spoofed source. The in-XDP implementation reproduces the RFC reference test vectors exactly.

Bounded reflection (TC-slip)

For a 1-in-k fraction of over-budget unknown queries, Segfault emits a truncated (TC=1) reply from XDP via XDP_TX, telling a real resolver to retry over TCP — which proves its address. The danger of reflecting toward a spoofed victim is bounded by two properties:

$$reflected\ rate \leq (1/k) \cdot \lambda_flood\ , \qquad amplification\ A = bytes_out\ /\ bytes_in \leq 1$$

because the truncated reply carries no answer section and is no larger than the query. A naive responder has A ≫ 1 (classic amplification); slip caps reflected volume at 1/k *and* removes amplification. This *bounds* reflection; it does not eliminate it. Lab setting: k = 2.

Adaptive controller

With admitted rate a, blocked rate b, capacity C and response ratio ρ = 1 − servfail/resp, define load L = a/C and drop-fraction φ = b/(a+b). The insight behind the adaptive hardening is that dropping most offered traffic is itself proof of a volumetric flood, even when ρ ≈ 1:

$$busy = (L \geq 0.7) \vee (\varphi > 0.2) \vee (\rho < 0.98)$$

$$under_attack = (\varphi > 0.6) \vee (\rho < 0.85)$$

CALM escalates to GUARD on busy, GUARD to SIEGE on under_attack; escalation is immediate, de-escalation requires 10 s of stability, so a pulsing attacker cannot ride the limiter up and down.

4.2 Implementation and kernel constraints

Making a parser plus a three-stage admission engine that the eBPF verifier accepts required three techniques:

- **Per-CPU scratch buffer** — the DNS name is copied into a fixed 288-byte per-CPU buffer with bounded loads (bpf_xdp_load_bytes), so the parser walks a buffer of known length rather than chasing packet pointers the verifier cannot bound.
- **Global (non-inlined) functions** — factoring the parser and slip builder into global functions caps the per-call state the verifier tracks (kernel 6.8 is stricter here than later kernels).
- **Explicit index masking** — every offset derived from packet data is masked to a hard bound (e.g. l3 &= 0x3f; dns &= 0xff), so a value reloaded from the stack keeps provable bounds; the EDNS cookie walk is bounded to the first 256 bytes.

The program attaches in native (driver) XDP where the NIC allows, falling back to generic (SKB) mode otherwise. On the Hyper-V hv_netvsc driver, native XDP requires LRO/GRO disabled (ethtool -K eth1 lro off gro off). Attachment is made idempotent by detaching any prior program in all three modes before attaching.

4.3 Admission pseudocode

```
fn admit(pkt, cfg, state) -> verdict:
    p = parse_bounded(pkt)                # Eth/VLAN/IP/UDP/DNS, per-CPU buffer
    if p is None or p.dport != 53: return PASS
    tier = lookup_tier(p.src)             # T2 proven / T1 returning / T0 unknown
    if tier == T2 or cookie_valid(p, cfg.secret):
        if gcra(state.pool[T2], 1): return PASS      # reserved pool
    w = cost(p, cfg)                      # qtype, DO bit, zone NXDOMAIN multiplier
    if tier == T0:
        if sketch_estimate(p.src) > cfg.heavy: return slip_or_drop(p, cfg)
    ok  = gcra(state.pool[tier], w)       # tier pool budget
    ok &= gcra(state.src[p.src],  w)       # per-source budget (T1/T2)
    ok &= gcra(state.pfx[p.prefix], w)     # per-/24 or /48 budget
    if ok: return PASS
    return slip_or_drop(p, cfg)           # 1-in-k -> XDP_TX TC=1, else XDP_DROP
```

5. Performance Evaluation and Comparison

5.1 Methodology

Two Hyper-V virtual machines on an isolated switch: a target (Ubuntu 24.04, kernel 6.8, native XDP, Unbound resolver + Segfault on eth1) and a generator (Kali) running the flood and a legitimate prober (200 q/s). Five baselines were compared under identical conditions: none (no filter), nftables (prerouting), nftables-netdev (ingress hook), fixed-xdp (fixed-threshold XDP — the rival design), and segfault. Each (baseline, scenario) pair was repeated three times; the per-field median is reported.

Because a real resolver falls back to TCP, success rate alone saturates near 1.0 for almost any defence; the discriminating metrics are therefore **tail latency (p99)** and **completed legitimate queries**. Throughput, CPU usage, and packet drops are measured as PS-022 requires.

5.2 IPv4 results — all six scenarios (median of 3)

Legitimate queries completed / p99 latency. Higher queries and lower p99 are better.

Scenario	none	nftables	nft-netdev	fixed-xdp	segfault
single	557 / 208 ms	574 / 202 ms	574 / 203 ms	1085 / 202 ms	3712 / 0.75 ms
pulse	758 / 210 ms	574 / 202 ms	574 / 202 ms	1080 / 202 ms	3707 / 0.71 ms
distributed	116 / 414 ms	53 / 529 ms	28 / 594 ms	119 / 544 ms	3719 / 1.13 ms
adaptive	180 / 422 ms	38 / 620 ms	60 / 451 ms	65 / 432 ms	3670 / 1.10 ms
prefix-spoof	0	0	0	0	3729 / 0.42 ms
water-torture	113 / 392 ms	49 / 469 ms	52 / 592 ms	68 / 680 ms	3684 / 1.11 ms

Segfault held 100% goodput in every scenario while absorbing 7–12 million packets per run. The sharpest result is **prefix-spoof**: when the attacker forges sources inside the client's own /24, every baseline drops the real client to zero completed queries, while Segfault served 3,729 — because it admits on proof, not prefix. Against the best rival design (fixed-xdp), Segfault completed 30–56× more legitimate queries under the multi-source scenarios.

5.3 IPv6 results (median of 3)

Scenario	none	nftables	nft-netdev	fixed-xdp	segfault
distributed	1242 / 204 ms	97 / 208 ms	97 / 208 ms	94 / 205 ms	3499 / 12.7 ms
water-torture	2722 / 2.6 ms	96 / 208 ms	97 / 208 ms	96 / 208 ms	3680 / 1.4 ms
prefix-spoof	0	0	0	0	up to 246 / 0.92 gp

Segfault is the only defence that keeps an IPv6 client alive under a spoofed flood. ***Honest note on prefix-spoof:*** this is the hardest case. With a correctly configured IPv6 TCP resolver and the flood held below the lab's virtual-switch limit, Segfault protects the client (up to 0.92 goodput at 5µs, while every baseline stays at 0), but establishment is intermittent: the slip-to-TCP nudge that makes IPv4 fast is currently IPv4-only, so over IPv6 the client relies on slower UDP timeouts to trigger the TCP proof. IPv6 slip support is the top future-work item; IPv6 water-torture and distributed are already strong.

5.4 Water-torture at three realism levels (segfault vs best baseline)

To test robustness, the random-subdomain flood was run against a local-NXDOMAIN resolver, a resolver that recurses to a local authoritative, and one with 50 ms of upstream latency added.

Resolver realism	segfault (queries / p99)	best baseline (queries / p99)
Local NXDOMAIN	3684 / 1.11 ms	113 / 392 ms
Recursive (loopback)	3511 / 1.9 ms	113 / 446 ms
Recursive + 50 ms upstream	3706 / 1.58 ms	129 / 526 ms

The win is robust to how realistic the attack is made — Segfault sheds the flood at the NIC before the resolver's single-threaded intake bottleneck, so making each query more expensive bends the baselines further (tail latency rose from 392 ms to 526 ms) without touching Segfault.

5.5 Result integrity

A result as clean as "100% goodput, sub-millisecond tail" was verified four ways to rule out measurement error:

- **A failing control.** The no-defence baseline visibly degrades under the identical attack (e.g. 294 ms, or 0 under prefix-spoof) — so the attack is real and the win is genuine.
- **No allow-list.** Datapath counters show pass_allow = 0; the client earns protection through the real proof path (proofs = 1, tcp_resp = 1 → pass_t2 = 6417, all answered).
- **Counters reconcile exactly.** Over one adaptive run the unknown tier saw 13,142,956 queries = 9,285,073 (pool drop) + 3,835,672 (heavy drop) + 64 (prefix) + 22,147 (admitted). Only 0.17% of the flood was admitted; the 6,560,403 slips equal ~1/2 of drops, matching k = 2.
- **Full offered load.** The prober sent its full ~3,700 queries per run — "100% goodput" is of a real workload, not of a trickle.

5.6 Throughput ceiling

Measured

A flood ramp (single-source, up to 8 Mpps requested) against Segfault plateaued at ~900 kpps of achieved rate — the Hyper-V virtual switch's limit — at which point Segfault sustained 100% goodput using only ~13% of one CPU core (85–88% idle), with no change in p99. Doubling generator threads did not raise the rate, confirming the wall is the virtual switch, not Segfault. In short, the lab never reached Segfault's limit.

Theoretical (server-grade)

From the measured operating point (900 kpps at 13% of one core) the per-packet cost is **~290 cycles/packet** — lean for a parser plus several map lookups and a CAS. The per-core ceiling is frequency-independent at ~6.9 Mpps (conservative); a ~3 GHz server core sheds ~10 Mpps. The ceiling is min(line-rate, cores · f/c):

Server NIC	Line-rate cap (108 B pkts)	Cores to reach it	Segfault ceiling
10 GbE	11.6 Mpps	~1–2	11.6 Mpps (line rate)
25 GbE	29 Mpps	~3	29 Mpps (line rate)
100 GbE	116 Mpps	~12	116 Mpps (line rate)

On server hardware Segfault is line-rate-bound: it drops a full-line-rate DNS flood using a fraction of the server's cores — the theoretical maximum any in-kernel defence can achieve. This matches production XDP deployments (e.g. Cloudflare L4Drop, Meta Katran) that drop at 10–100 GbE line rate on a few cores.

In-kernel efficiency

A separate in-kernel microbench (pktgen to a veth pair, with no virtual switch in the path) measured the raw drop-path throughput directly. A minimal do-nothing XDP_DROP sheds 2.52 Mpps using 16% of one core; Segfault's full pipeline — bounded parse, trust-tier lookup, cost-weighting and three-stage GCRA admission — sheds 2.32 Mpps at 21%, i.e. only about 40% more CPU per packet than a do-nothing drop. Neither program was CPU-saturated (the generator was the limit), so the true ceiling is higher still. For scale, a representative rival design reports ~0.21 Mpps at comparable relative efficiency — roughly one tenth the absolute rate on its rig.

5.7 Comparison with alternative approaches

Approach	Strength	Structural limitation
BIND RRL	Mature per-prefix response limiting	Acts after processing; weak vs random-subdomain; no proven-client notion
iptables/nftables hashlimit	General, well understood	Runs after socket-buffer allocation; defeated by adaptive/spoofed sources; no cost model
nftables netdev/ingress	Earlier, cheaper hook	Still no trust tiers, no proof, no cost model
dnsdist	Flexible userspace proxy	Userspace per-packet cost; not kernel-bypass line rate
Fixed-threshold XDP (rivals)	Very fast; drops early	Single scalar (rate) is what the attacker controls; cannot tell proven from spoofed; punishes prefix-spoof victims
Segfault (this work)	**XDP line rate + trust-by-proof + cost weighting + bounded slip + adaptive controller**	**Needs native-XDP NIC for best results; proven tier costs a one-time round-trip; single-host (not anycast scrubbing)**

5.8 Honest scope and limitations

- Laboratory, not the open Internet: two Hyper-V VMs, synthetic generator at ~0.2–0.9 Mpps; this demonstrates the mechanism, not silicon-limit throughput.
- Below link saturation: Segfault protects server CPU and service, not the physical link; link-saturating volumetric attacks are an upstream/anycast problem.
- Proven tier has a cost and a TTL: a first-contact UDP client pays one round-trip before promotion, and proven status expires after a TTL.
- Reflection is bounded ($\leq 1/k$), not eliminated; random-subdomain detection is heuristic (zone NXDOMAIN ratio), not universal.
- IPv6 prefix-spoof recovers to goodput 0.92 once the resolver answers over IPv6 TCP, but is intermittent: the slip-to-TCP nudge that makes IPv4 fast is currently IPv4-only, so IPv6 legitimate sources fall back on slower UDP retransmit timeouts. IPv6 slip support is the top future-work item.

6. References

[1] BPF and XDP documentation — The Linux Kernel. https://docs.kernel.org/bpf/

[2] T. Høiland-Jørgensen et al., “The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel,” ACM CoNEXT, 2018.

[3] libbpf / libbpf-rs — BPF CO-RE loader library. https://github.com/libbpf/libbpf

[4] O. Sury, W. Toorop, D. Eastlake, M. Andrews, “Domain Name System (DNS) Cookies,” RFC 9018, 2021.

[5] J.-P. Aumasson, D. J. Bernstein, “SipHash: a fast short-input PRF,” 2012.

[6] G. Cormode, S. Muthukrishnan, “An improved data stream summary: the Count-Min sketch,” J. Algorithms, 2005.

[7] ATM Forum, “Traffic Management Specification 4.0” (Generic Cell Rate Algorithm), 1996.

[8] Cloudflare, “L4Drop: XDP DDoS mitigations,” Cloudflare blog, 2018.

[9] Meta (Facebook), “Katran: a high-performance layer-4 load balancer,” 2018.

[10] Unbound recursive DNS resolver — NLnet Labs. https://nlnetlabs.nl/projects/unbound/

7. Conclusion

Segfault answers PS-022 by moving the question from “how fast is this source?” to “who is this, and what does the query cost?”, and by enforcing that answer at the earliest point in the kernel. Across six attack scenarios on IPv4, three on IPv6, and a water-torture flood tested at three realism levels, a legitimate client stayed at its no-attack latency while every baseline — including a fixed-threshold XDP limiter representative of rival designs — collapsed to hundreds of milliseconds and single-digit query counts, dropping to zero under prefix-spoof. The result was verified with a failing control and exact counter reconciliation, and the datapath's ~290-cycle per-packet cost places it in the line-rate regime of production XDP systems.

Next steps: harden the IPv6 proven-tier path under same-prefix spoofing; add automatic capacity calibration under live load; and evaluate on server-class hardware with a multi-queue NIC to measure the true saturation point. The design, the measurements, and the honest accounting of scope together make Segfault a complete, defensible answer to the problem statement.